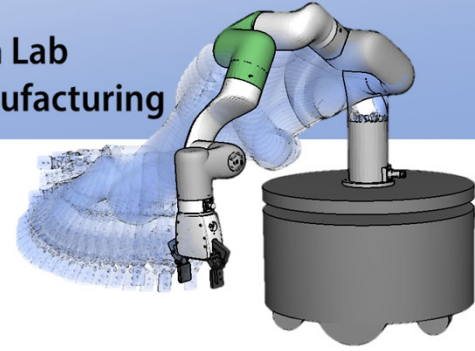


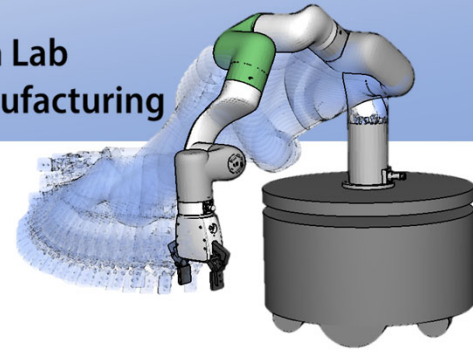


# 制御構造とプログラムの図示

原田研究室 准教授 万 偉偉

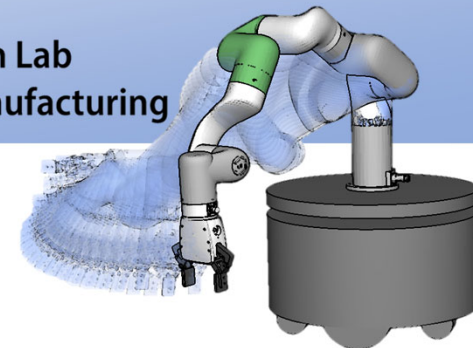
基礎工学部 F522号室

[wan@sys.es.osaka-u.ac.jp](mailto:wan@sys.es.osaka-u.ac.jp)



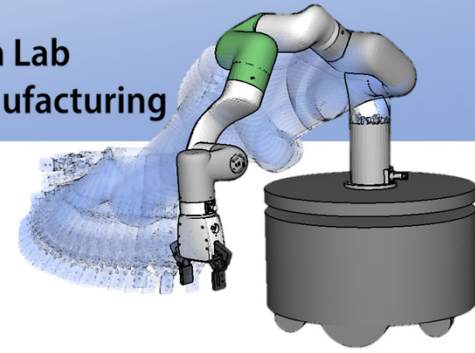
# 復習

- アルゴリズムとは
  - 問題を解くための論理または手順
- プログラミングとは
  - データ構造とアルゴリズム
  - 良いプログラム、計算方法の選択
  - 事前条件・事後条件・不変条件
  - 自然言語・疑似コーディング・Cプログラム
- データ型
  - 文字・整数・実数・配列・構造体・共用体・ポインタ
- データ構造
  - リスト、待ち行列・スタック・木・ハッシュ表・グラフ
  - データ構造+操作の手法 → 抽象データ型
- 演算子
  - 優先順位



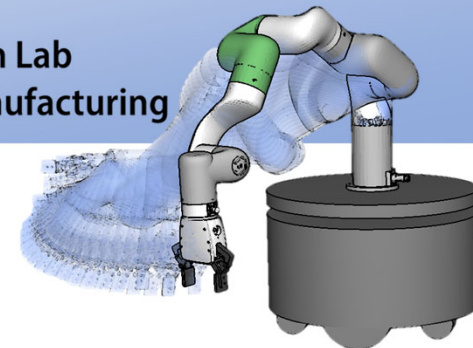
# 本日の内容

- フローチャート
- NSチャート
- プログラムの制御構造  
(構造化プログラミング)
- 擬似言語 (擬似コード, PDL)
- 拡張バックス記法
- ハイレベル



# 計算機言語の制御構造

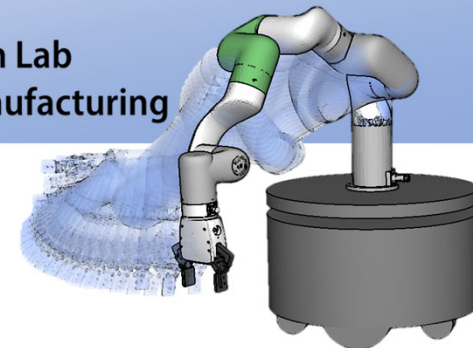
- 連接構造
- 選択構造
  - 単一選択構造(if ~ then ~ end if)
  - 分岐選択構造(if ~ then ~ else ~ end if)
  - 複数選択構造(switch ~ end switch)
- 反復構造
  - 前判定反復構造(while ~ end while)
  - 後判定反復構造(do ~ until)
  - カウンタ制御反復構造(for)



# 各制御構造の図示

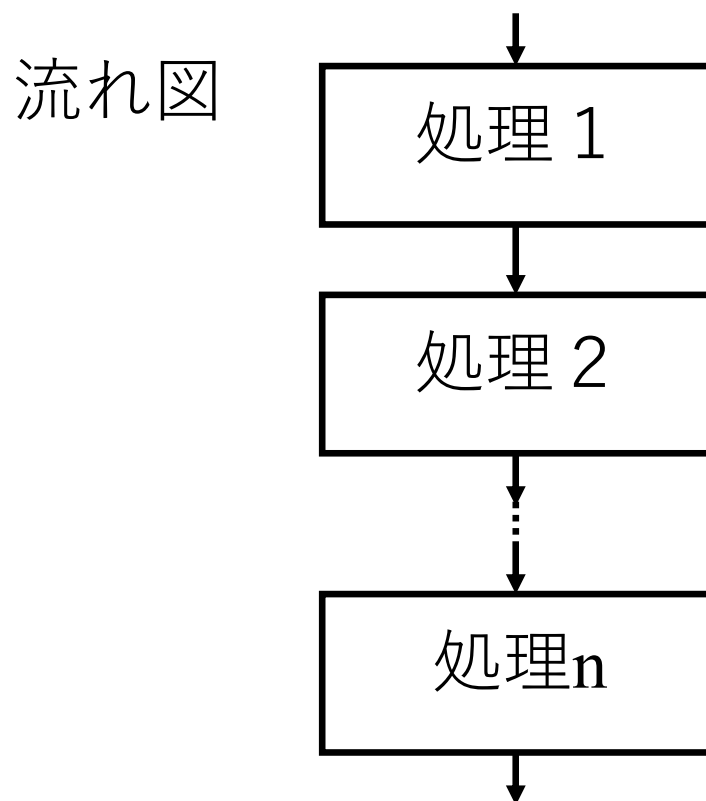
- フローチャート（流れ図, Flow Chart）
  - 論文や説明書提案書の作成によく使われる
- NSチャート（Nassi-Schneiderman Chart）





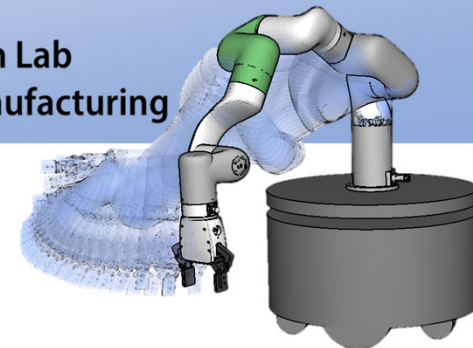
# 連接構造

- 処理 1 , 処理 2 ... 処理 $n$ の逐次実行
- このようなまとまりをブロック(block)という.



NSチャート

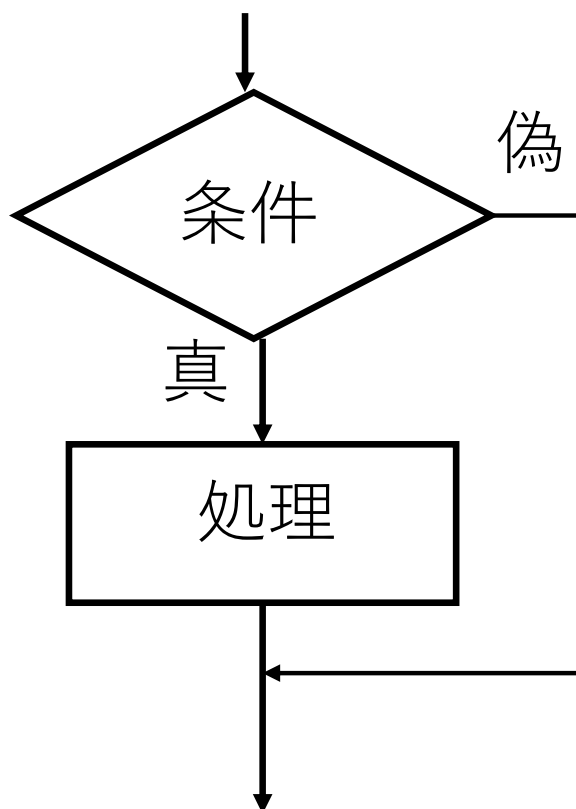




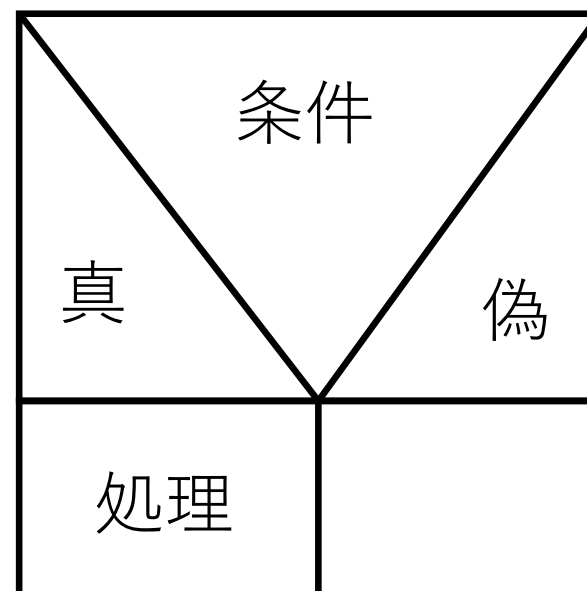
# 単一選択構造

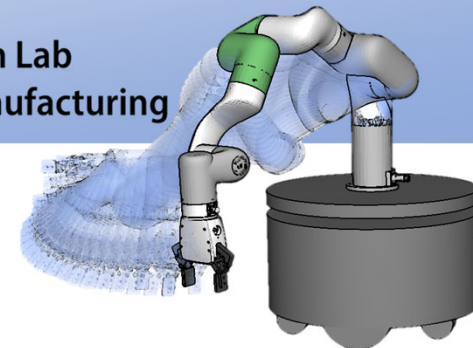
- 条件が真のときに処理を行う
- C言語：if (条件) 処理;

流れ図



NSチャート

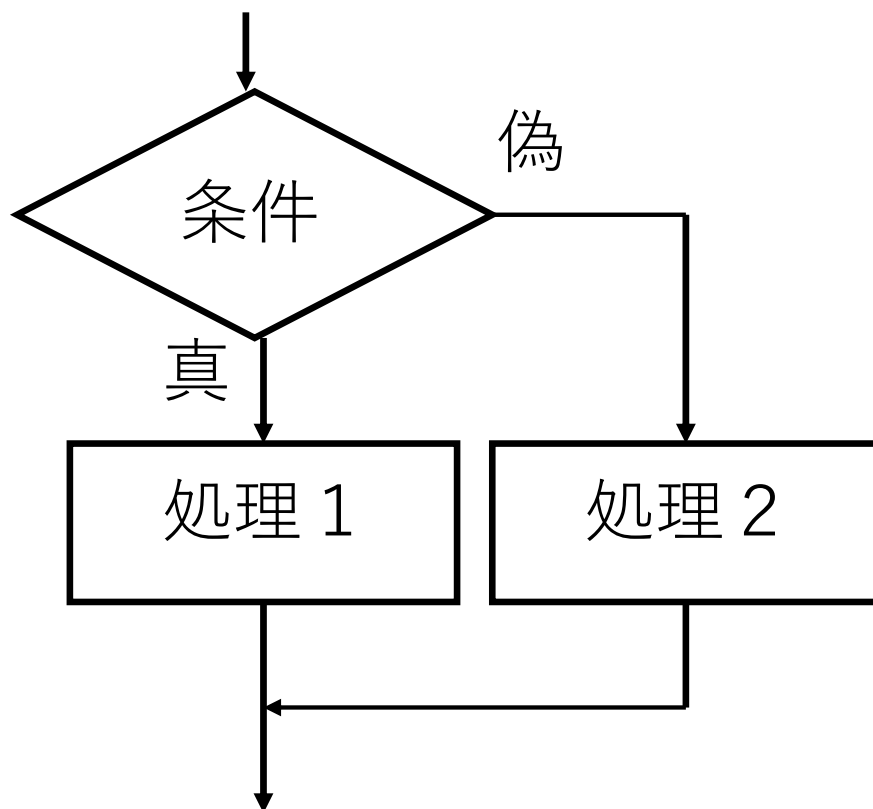




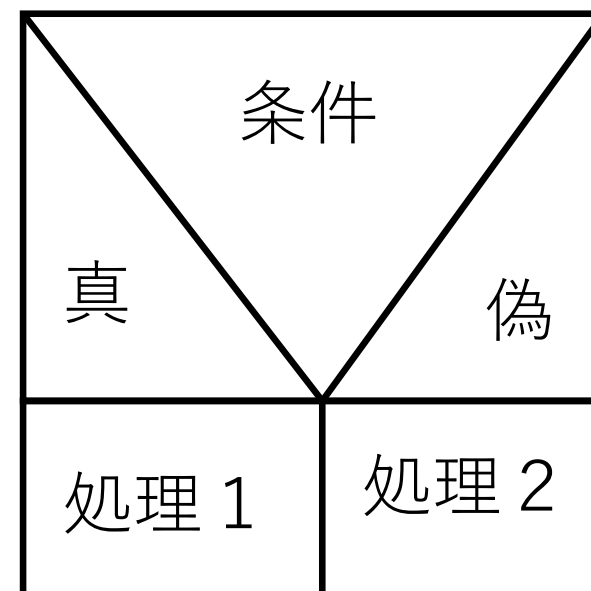
# 分岐選択構造

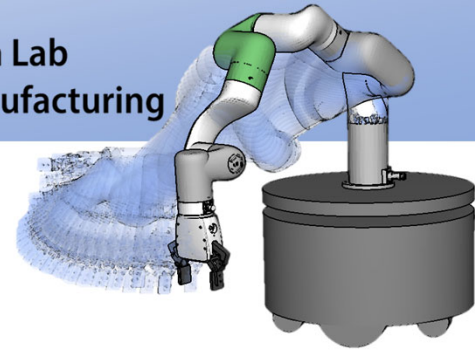
- C言語 : `if (条件) then 処理 1 ; else 処理 2 ;`

流れ図



NSチャート

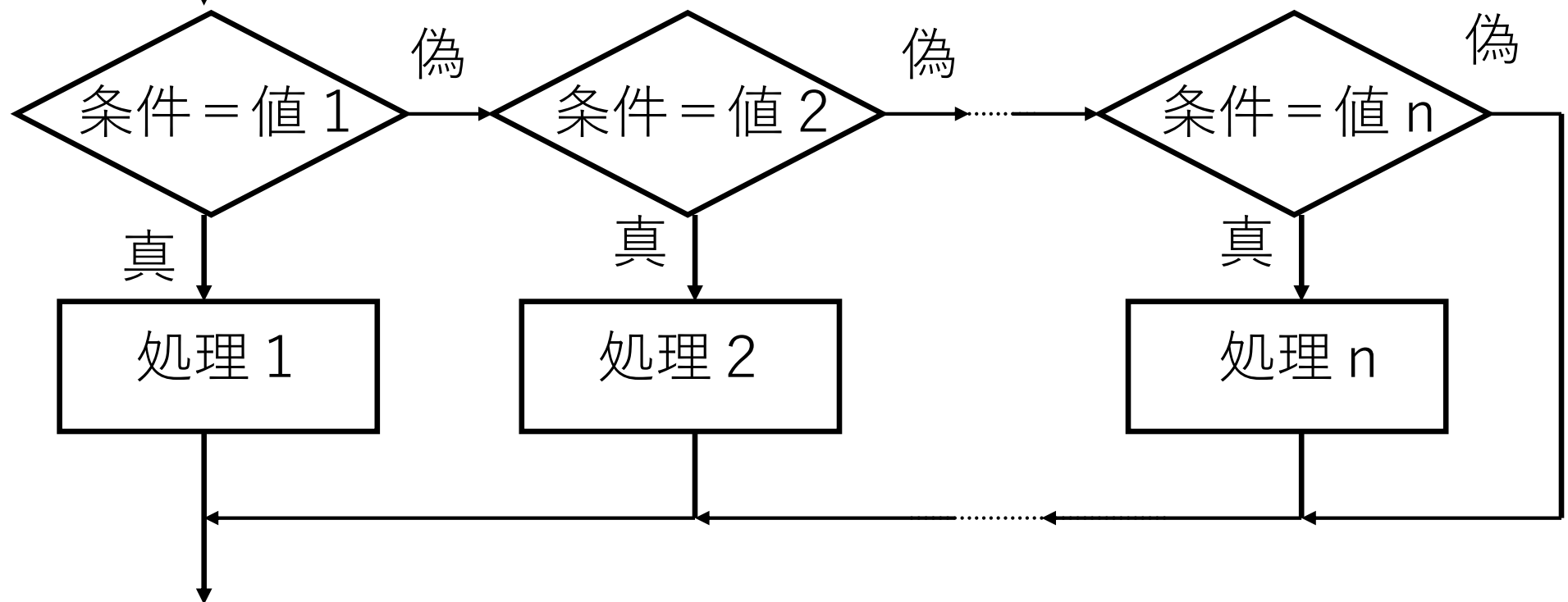


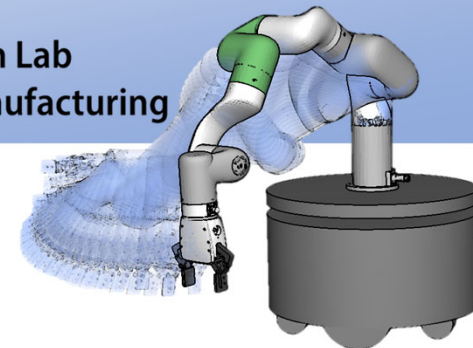


# 複数選択構造

- C言語 : `switch (条件) { case 値1: 処理 1 ; ... };`

流れ図



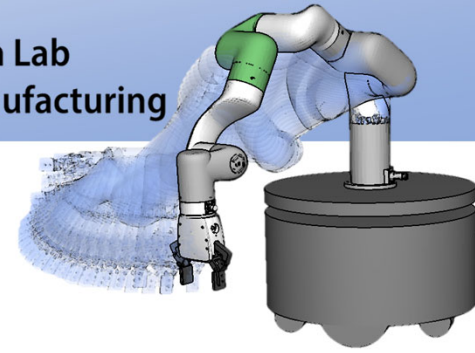


# 複数選択構造

- C言語 : `switch (条件) { case 値1: 処理 1 ; ... };`

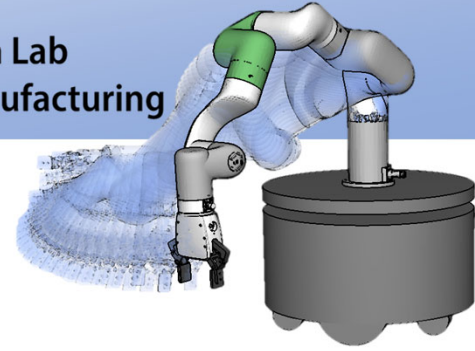
NSチャート

| 条件   |      |     |      |
|------|------|-----|------|
| 値 1  | 値 2  | ... | 値 n  |
| 処理 1 | 処理 2 | ... | 処理 n |



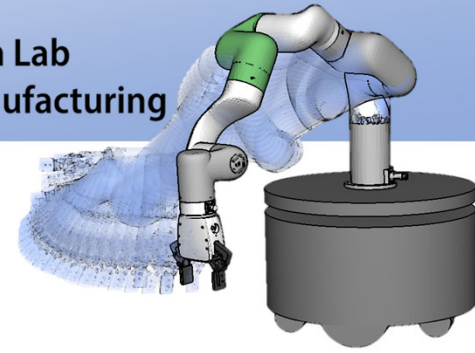
# 前判定反復構造

- 繰り返しの**前に**条件を判定する
- C言語 : while (条件) 処理;



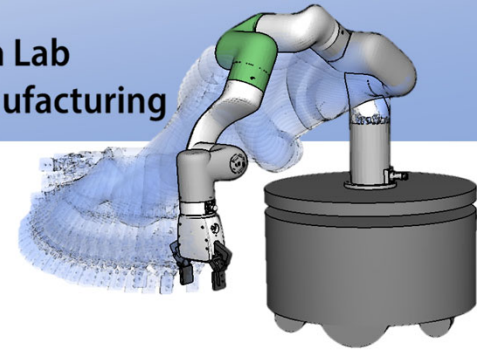
# 後判定反復構造

- 繰り返しの後に条件を判定する
- C言語 : do 処理 while (条件);



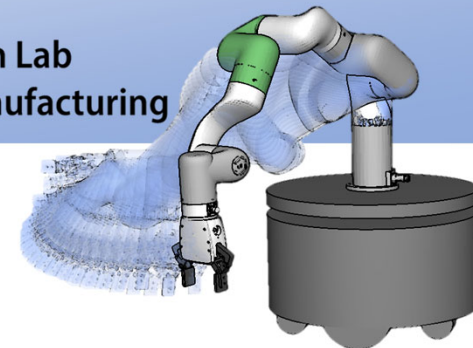
# カウンタ制御反復構造

- 制御変数（カウンタ変数ともいう）
- 制御変数の初期化处理
- 制御変数の繰り返し 1 回あたりの増加処理（減少処理）
- 制御変数の終了値を判定する終了条件
- C言語：for (初期化处理; 終了条件; 増加処理) 処理;
- int i;
- for (i = 0; i < 10; i++) {} -> While



# カウンタ制御反復構造



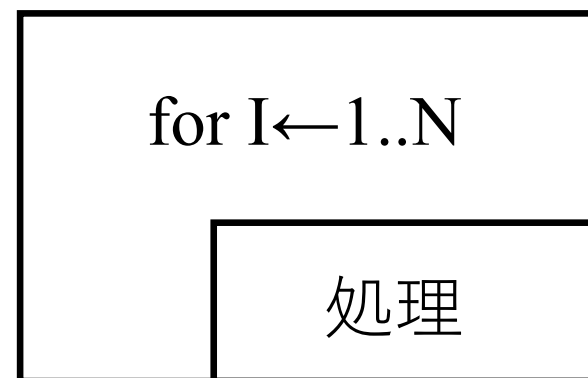


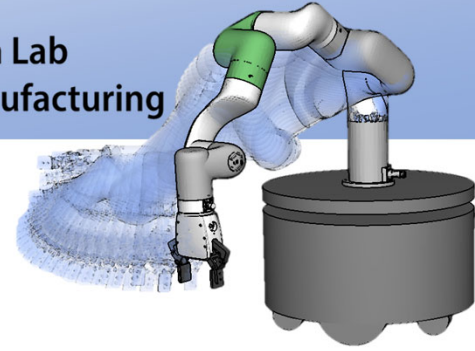
# カウンタ制御反復構造

NSチャート



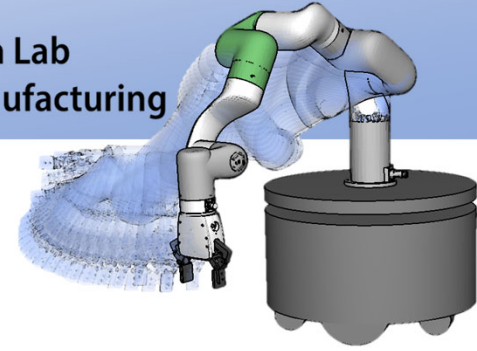
NSチャート





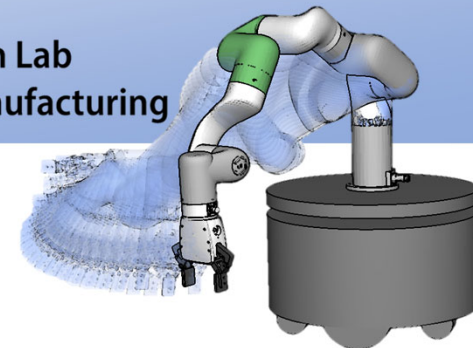
# フローチャートの利点と欠点

- アルゴリズムの理解に補助となる**標準シンボル**
- おおまかなプログラムの流れを表現するには良いツール
- プログラムの完全な仕様記述にフローチャートは向かない（時間と紙の無駄）



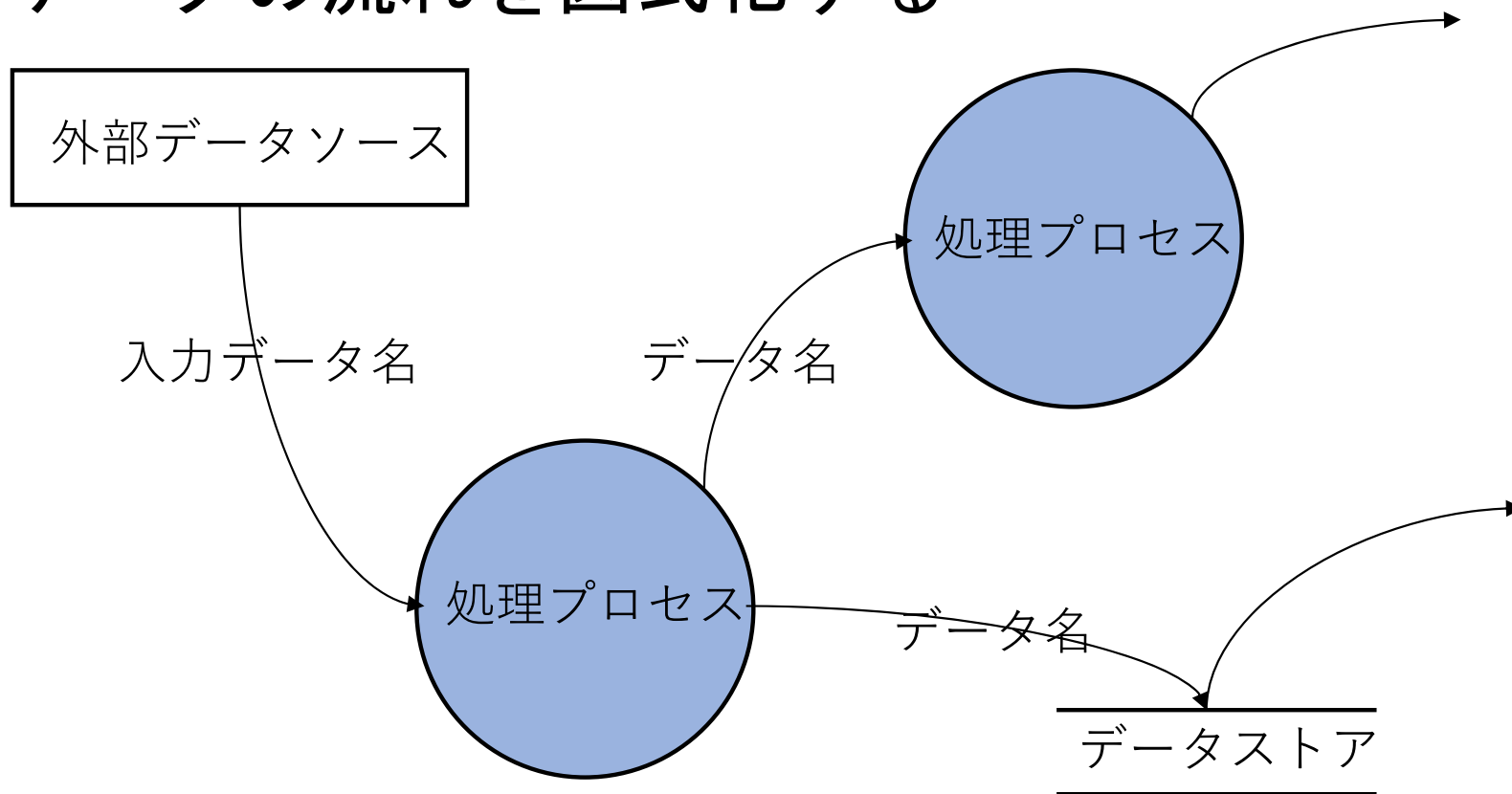
# NSチャートの利点と欠点

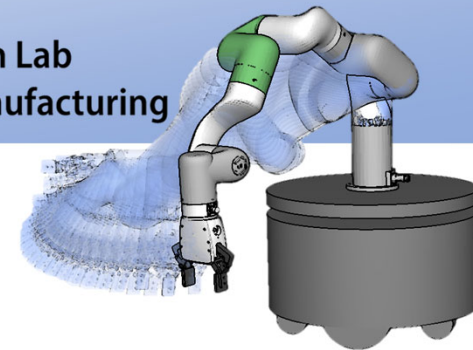
- 慣れれば、フローチャートより直感的に理解できる.
- 反復構造中の**例外処理**(break, continue)を上手く表現できない.



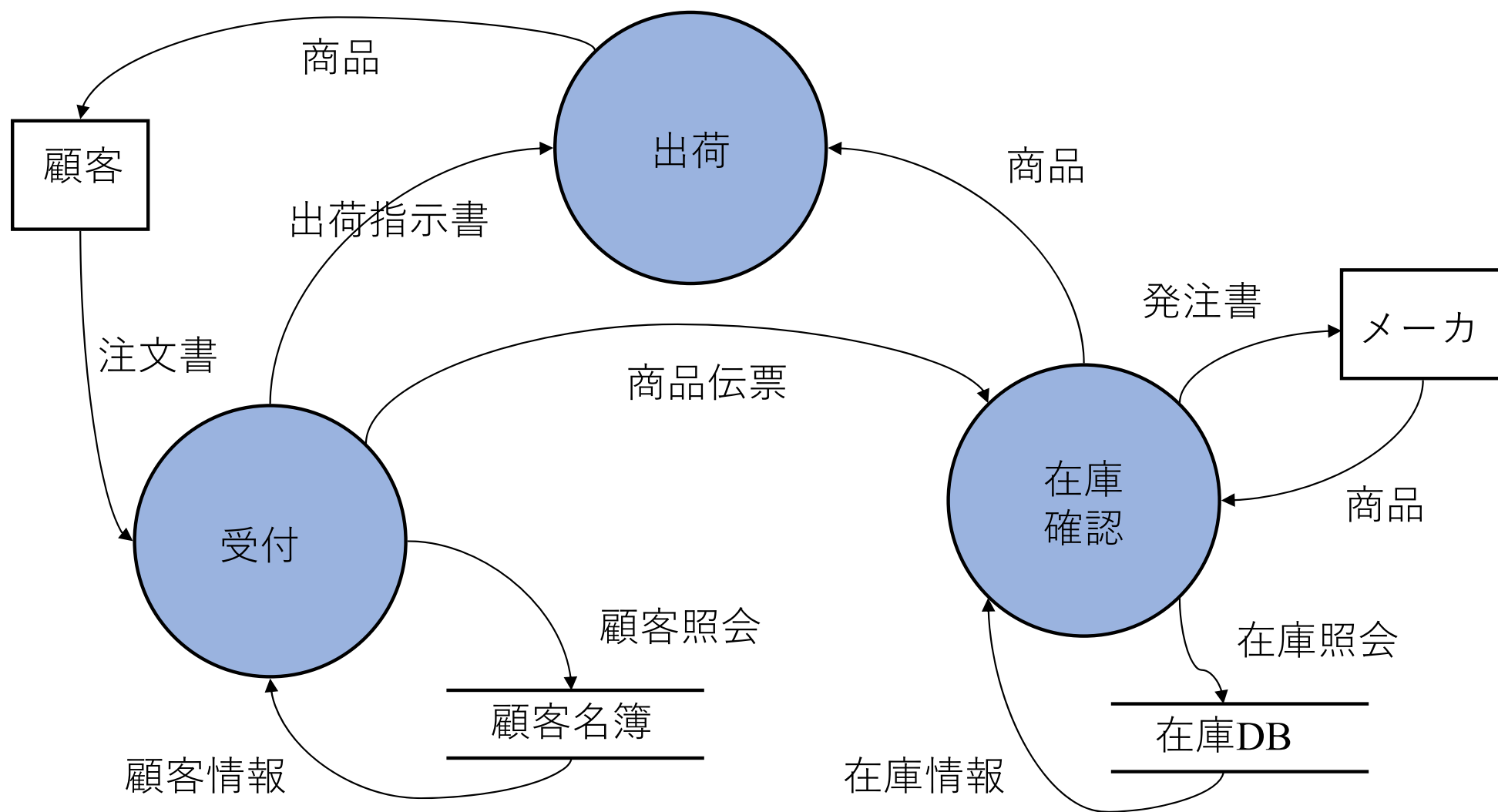
# データフロー図

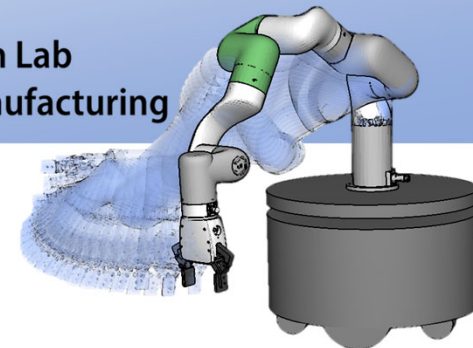
- アルゴリズム以上のもの
- データの流れを図式化する





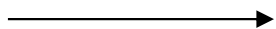
# データフロー図の例



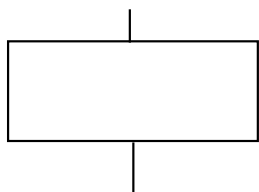


# フローチャート詳細

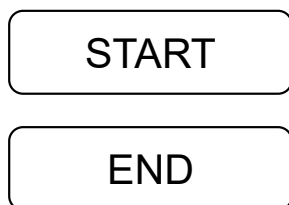
流れ線



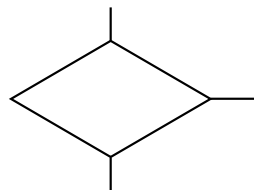
処理



端子



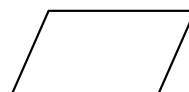
判断



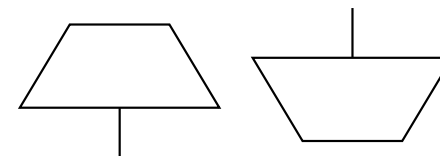
結合子



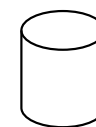
入出力



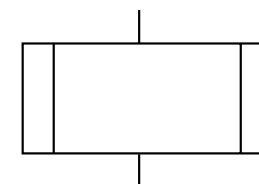
繰り返し

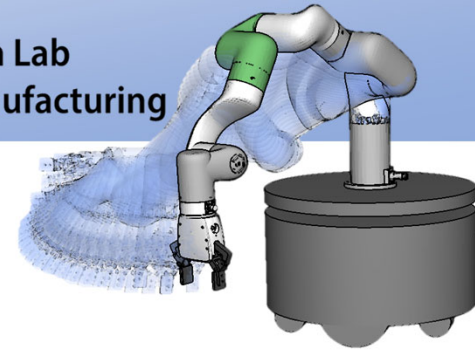


磁気ディスク



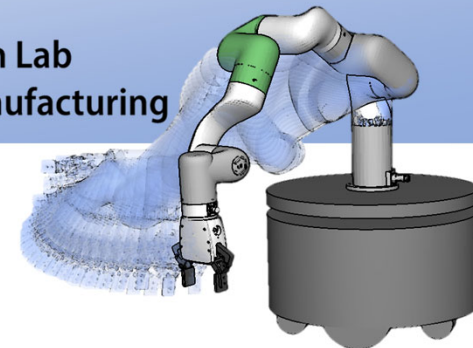
定義済み処理





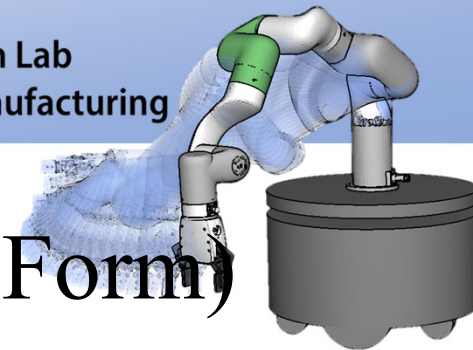
# 擬似言語とは

- 擬似言語
  - 擬似コード, PDL(Program Description Language)とも呼ぶ. 基本的な語彙や構文だけを約束しておき, 細部については自由に誤解を生じない程度に表現.



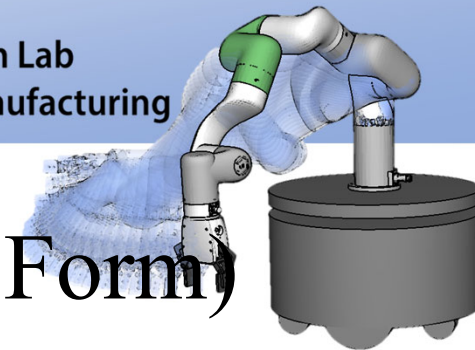
# 擬似言語の基本

- 基本処理（入力・計算・出力）
  - 入力                      read 変数名
  - 計算                      変数名 ← 変数(数値) / 変数(数値)
  - 出力                      write 変数名
- コンピュータ言語はよくBNFで記述される
- 擬似言語についてもBNFで記述してみましょう
- 本授業における記述形式の取り決め(Backus-Naur Form, BNF)
  - ::=    定義
  - <>    任意の表現を許す
  - {}    0回以上の出現
  - |    あるいは
  - 例えば
    - 10以内の整数    <digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9



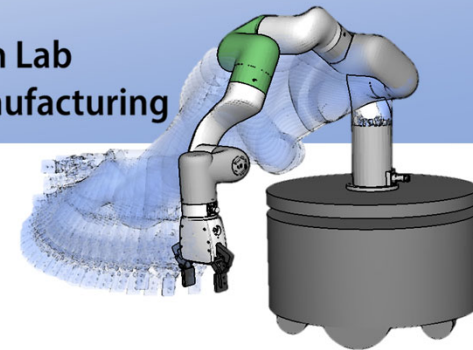
# バックス記法 (BNF: Backus-Naur Form)

- 言語を定義するためのメタ言語
  - メタ言語 言語を記述する言語
  - メタデータ データを記述するデータ
    - Music data ? 音声・曲 metadata? 作曲者・演奏者
- プログラミング言語やデータ記述言語の文法決めるを行う
- 表記（どのような文字を使うか）についての定義を行うためのものであり，演算や言語の機能まで定めるものではない



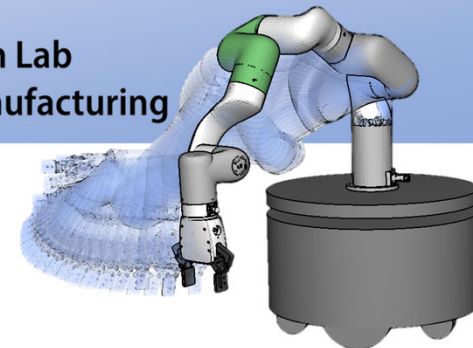
# バックス記法 (BNF: Backus-Naur Form)

- バックス記法 (BNF: Backus-Naur Form)
  - $\langle X \rangle$  は,  $X$  が表す記述を表現
  - $::=$  は左辺の記述が右辺の記述で表現される
  - $|$  はこの左右のいずれかの記述が可能
  - $|$  で区切られていない複数の記述は, 記述順に並べて表す
  - $\langle \rangle$  でくくられていない表現は, その表現のまま記述 (ダブルクォーテーションやシングルクォーテーションを使う流儀もある)



# バックス記法の歴史

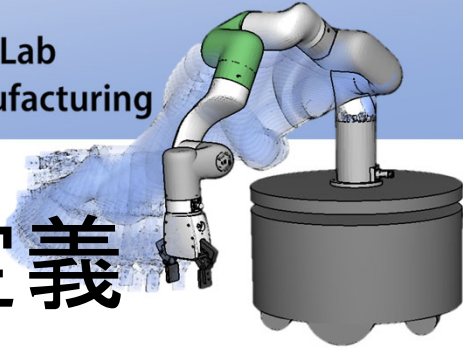
- John Backus and Peter Naurによる発明
- 最初の汎用プログラミング言語Algoal 58 / Algol 60の文法を定義するために開発され
  - Algoal 58/60  $\rightarrow$  CPL  $\rightarrow$  BCPL  $\rightarrow$  B  $\rightarrow$  C
  - Algoal: Algorithmic Language
  - CPL: Combined Programming Language
  - BCPL: Basic Combined Programming Language
  - B
  - C
- 最初の汎用プログラミング言語1958年、ACM提出



# バックス記法の例

- $\langle \text{assignment statement} \rangle ::= \langle \text{variable} \rangle = \langle \text{expression} \rangle$
- $\langle \text{number} \rangle ::= \langle \text{digit} \rangle \mid \langle \text{number} \rangle \langle \text{digit} \rangle$
- $\langle \text{digit} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

設問



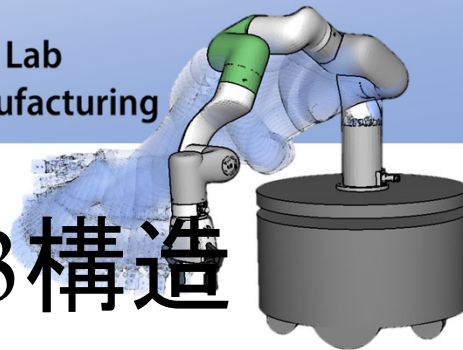
# バックス記法の例 再帰的な定義

- $\langle \text{二進数} \rangle ::= ?$

設問

- $\langle \text{変数の名前} \rangle ::= ?$

設問



# BNFで定義する擬似言語の基本3構造

- 連接構造

- $\langle \text{文} \rangle$
- $\{ \langle \text{文} \rangle \}$

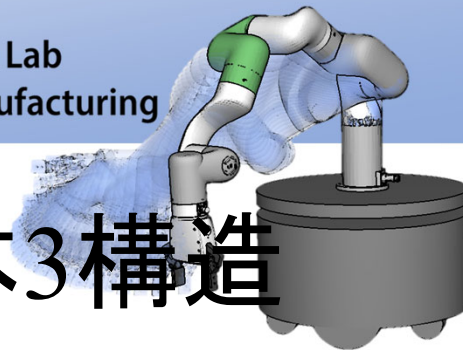
- 分岐選択構造

```
if  $\langle \text{条件式} \rangle$  then  
     $\{ \langle \text{文} \rangle \}$   
else  
     $\{ \langle \text{文} \rangle \}$   
endif
```

例) read A  
B $\leftarrow$ A+1  
write B

例) if weight > 70 then  
weight $\leftarrow$ weight-10  
else  
weight $\leftarrow$ 60  
endif

例：成績xが60点以上なら「合格」と出力。  
60点未満なら「不合格」と出力。



# BNFで定義する擬似言語の基本3構造

- 前判定反復構造

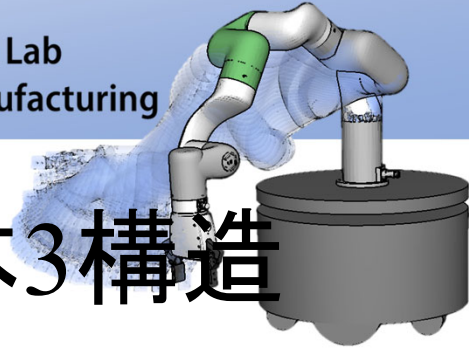
```
while <条件式> do  
    {<文>}  
endwhile
```

例) while sum<100 do  
 point←point+2  
 sum←sum+point  
endwhile

- 後判定反復構造

```
do  
    {<文>}  
while <条件式>
```

例：キーボードからxに整数値  
を入力させる。yにxを加える。  
xが0以下なら終了する。



# BNFで定義する擬似言語の基本3構造

- カウンタ制御反復構造

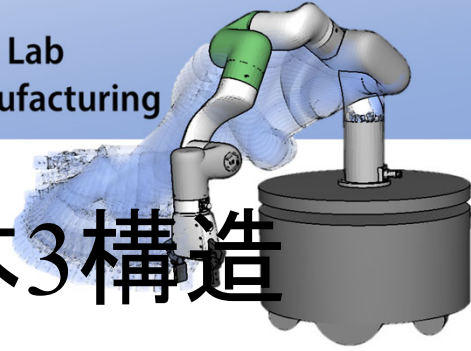
設問

例：5の階乗を求める．値は変数 $r$ に格納する．カウンタ変数は $i$ を用いる．

```
for i = 0 to 5 do
```

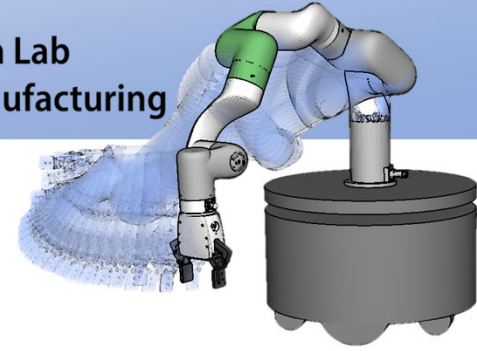
```
   $r = r * i$ 
```

```
endfor
```



# BNFで定義する擬似言語の基本3構造

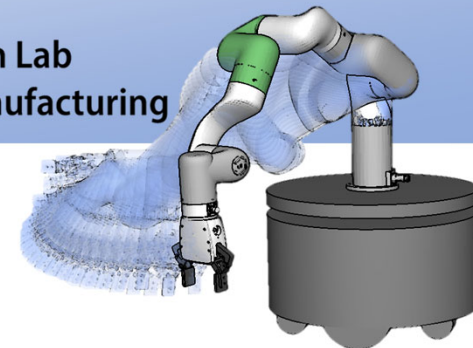
- メソッド呼び出し
  - $\langle \text{関数名} \rangle ( \langle \text{文字列} \rangle )$       例) `sampleFunc(x)`



# 擬似言語で変数を宣言

- 変数宣言，定義については，明確な規則があるわけではないが，一般的には下記のように宣言する
  - 整数 (int)，実数 (float)，文字 (char)，文字列 (string)，論理値 (boolean)
  - 配列 変数名[ ]

例)     int a  
          float b[100]



# まとめ

- フローチャート
- NSチャート
  - データフロー
- プログラムの制御構造  
(構造化プログラミング)
- 擬似言語 (擬似コード, PDL)
- 拡張バックス記法